In [3]:
```
import pandas as pd

# Load data
df = pd.read_excel("Startup_Financial_KPI_Sample.xlsx")  # or use pd.read_csv("..."
```

In [5]:
```
df["Burn Rate"] = df["Total Expenses"] - df["Revenue"]
df["Run Rate"] = df["Revenue"].iloc[-1] * 12
df["CAC"] = df["Marketing Spend"] / df["New Customers"]
df["ARPU"] = df["Revenue"] / df["Total Customers"]
df["Churn Rate"] = df["Churned Customers"] / df["Total Customers"]
df["Customer Lifetime"] = 1 / df["Churn Rate"].replace(0, pd.NA)
```

In [7]:
```
import pandas as pd

# Load your dataset
df = pd.read_excel("Startup_Financial_KPI_Sample.xlsx")  # Replace with your actual

# Calculate ARPU if 'Revenue' and 'Total Customers' columns exist
if 'Revenue' in df.columns and 'Total Customers' in df.columns:
    df['ARPU'] = df['Revenue'] / df['Total Customers']
    print("'ARPU' column calculated and added to the DataFrame.")
else:
    print("Cannot calculate 'ARPU' because 'Revenue' or 'Total Customers' column is

# Calculate Customer Lifetime if 'Churned Customers' and 'Total Customers' columns
if 'Churned Customers' in df.columns and 'Total Customers' in df.columns:
    df['Churn Rate'] = df['Churned Customers'] / df['Total Customers']
    df['Customer Lifetime'] = 1 / df['Churn Rate'].replace(0, pd.NA)
    print("'Customer Lifetime' column calculated and added to the DataFrame.")
else:
    print("Cannot calculate 'Customer Lifetime' because 'Churned Customers' or 'Tot

# Calculate LTV if 'ARPU' and 'Customer Lifetime' columns exist
if 'ARPU' in df.columns and 'Customer Lifetime' in df.columns:
    gross_margin = 0.80  # Assuming 80% gross margin
    df['LTV'] = df['ARPU'] * gross_margin * df['Customer Lifetime']
    print("'LTV' column calculated and added to the DataFrame.")
else:
    print("Cannot calculate 'LTV' because 'ARPU' or 'Customer Lifetime' column is m

# Save the updated DataFrame to a new Excel file
output_file_path = "Startup_Financial_KPI_Updated.xlsx"
df.to_excel(output_file_path, index=False)
print(f"Updated DataFrame saved to {output_file_path}")
```

```
'ARPU' column calculated and added to the DataFrame.
'Customer Lifetime' column calculated and added to the DataFrame.
'LTV' column calculated and added to the DataFrame.
Updated DataFrame saved to Startup_Financial_KPI_Updated.xlsx
```

In [9]:
```
import pandas as pd

# Load your dataset
df = pd.read_excel("Startup_Financial_KPI_Sample.xlsx")  # Replace with your actual
```

```
# Calculate CAC if 'Marketing Spend' and 'New Customers' columns exist
if 'Marketing Spend' in df.columns and 'New Customers' in df.columns:
    df['CAC'] = df['Marketing Spend'] / df['New Customers']
    print("'CAC' column calculated and added to the DataFrame.")
else:
    print("Cannot calculate 'CAC' because 'Marketing Spend' or 'New Customers' colu

# Calculate LTV:CAC if 'LTV' and 'CAC' columns exist
if 'LTV' in df.columns and 'CAC' in df.columns:
    df['LTV:CAC'] = df['LTV'] / df['CAC']
    print("'LTV:CAC' column calculated and added to the DataFrame.")
else:
    print("Cannot calculate 'LTV:CAC' because 'LTV' or 'CAC' column is missing.")

# Save the updated DataFrame to a new Excel file
output_file_path = "Startup_Financial_KPI_Updated.xlsx"
df.to_excel(output_file_path, index=False)
print(f"Updated DataFrame saved to {output_file_path}")
```

```
'CAC' column calculated and added to the DataFrame.
Cannot calculate 'LTV:CAC' because 'LTV' or 'CAC' column is missing.
Updated DataFrame saved to Startup_Financial_KPI_Updated.xlsx
```

In [11]:
```
df.to_excel("Startup_KPI_Report.xlsx", index=False)
```

In [13]:
```
print(df.columns.tolist())
['Month', 'Revenue', 'Total Expenses', 'Marketing Spend', 'New Customers', 'Total C
df.rename(columns={'ActualSignupColumnName': 'signup_date', 'ActualRevenueColumnNam
df.rename(columns={'ActualSignupColumnName': 'signup_date', 'ActualRevenueColumnNam
```

```
['Month', 'Revenue', 'Total Expenses', 'Marketing Spend', 'New Customers', 'Total Cu
stomers', 'Churned Customers', 'CAC']
```

In [15]:
```
import pandas as pd

# Example DataFrame
df = pd.DataFrame({'customer_id': [1, 2, 3]})

# Add 'signup_date' column with a constant value
df['signup_date'] = pd.to_datetime('2024-01-01')
```

In [17]:
```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({'customer_id': [1, 2, 3]})

# Assign a constant revenue date
df['revenue_date'] = pd.to_datetime('2025-04-23')

print(df)
```

```
   customer_id revenue_date
0            1   2025-04-23
1            2   2025-04-23
2            3   2025-04-23
```

In [19]:
```
import pandas as pd

# Display the column names
print(df.columns)
```

```
Index(['customer_id', 'revenue_date'], dtype='object')
```

In [23]:
```
# Assign a default date if 'signup_date' is missing
df['signup_date'] = pd.to_datetime('2025-04-23')

# Alternatively, derive 'signup_date' from another column if applicable
# df['signup_date'] = pd.to_datetime(df['some_other_column'])
```

In [ ]:
```
# Add 'revenue_date' as 30 days after 'signup_date'
df['revenue_date'] = df['signup_date'] + pd.DateOffset(days=30)
```

In [25]:
```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'customer_id': [1, 2, 3],
    'signup_date': ['2025-01-15', '2025-02-20', '2025-03-25']
})

# Convert 'signup_date' to datetime
df['signup_date'] = pd.to_datetime(df['signup_date'])
# Extract 'signup_month' as a Period (YYYY-MM)
df['signup_month'] = df['signup_date'].dt.to_period('M')
```

In [27]:
```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'customer_id': [1, 2, 3],
    'revenue_date': ['2025-01-15', '2025-02-20', '2025-03-25']
})

# Convert 'revenue_date' to datetime
df['revenue_date'] = pd.to_datetime(df['revenue_date'])
# Extract 'revenue_month' as a Period (YYYY-MM)
df['revenue_month'] = df['revenue_date'].dt.to_period('M')
```

In [29]:
```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'customer_id': [1, 2, 3],
    'signup_date': ['2025-01-15', '2025-02-20', '2025-03-25'],
    'revenue_date': ['2025-01-20', '2025-02-25', '2025-03-30'],
    'revenue': [100, 200, 300]
})

# Convert 'signup_date' to datetime
df['signup_date'] = pd.to_datetime(df['signup_date'])
```

```
# Extract 'signup_month' as a Period (YYYY-MM)
df['signup_month'] = df['signup_date'].dt.to_period('M')

# Convert 'revenue_date' to datetime
df['revenue_date'] = pd.to_datetime(df['revenue_date'])

# Extract 'revenue_month' as a Period (YYYY-MM)
df['revenue_month'] = df['revenue_date'].dt.to_period('M')

print(df[['customer_id', 'signup_date', 'signup_month', 'revenue_date', 'revenue_mo
```

```
   customer_id signup_date signup_month revenue_date revenue_month
0            1  2025-01-15      2025-01   2025-01-20       2025-01
1            2  2025-02-20      2025-02   2025-02-25       2025-02
2            3  2025-03-25      2025-03   2025-03-30       2025-03
```

In [31]:
```
cohort_data = df.groupby(['signup_month', 'revenue_month'])['revenue'].sum().reset_
print(cohort_data)
```

```
  signup_month revenue_month  revenue
0      2025-01       2025-01      100
1      2025-02       2025-02      200
2      2025-03       2025-03      300
```

In [33]:
```
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'customer_id': [1, 2, 3, 4],
    'signup_date': ['2025-01-15', '2025-01-20', '2025-02-10', '2025-02-15'],
    'revenue_date': ['2025-01-30', '2025-02-05', '2025-02-20', '2025-03-01'],
    'revenue': [100, 200, 150, 300]
})

# Convert 'signup_date' and 'revenue_date' to datetime
df['signup_date'] = pd.to_datetime(df['signup_date'])
df['revenue_date'] = pd.to_datetime(df['revenue_date'])

# Extract 'signup_month' and 'revenue_month' as Periods (YYYY-MM)
df['signup_month'] = df['signup_date'].dt.to_period('M')
df['revenue_month'] = df['revenue_date'].dt.to_period('M')

# Group by 'signup_month' and 'revenue_month' and sum the revenue
cohort_data = df.groupby(['signup_month', 'revenue_month'])['revenue'].sum().reset_

print(cohort_data)
```

```
  signup_month revenue_month  revenue
0      2025-01       2025-01      100
1      2025-01       2025-02      200
2      2025-02       2025-02      150
3      2025-02       2025-03      300
```

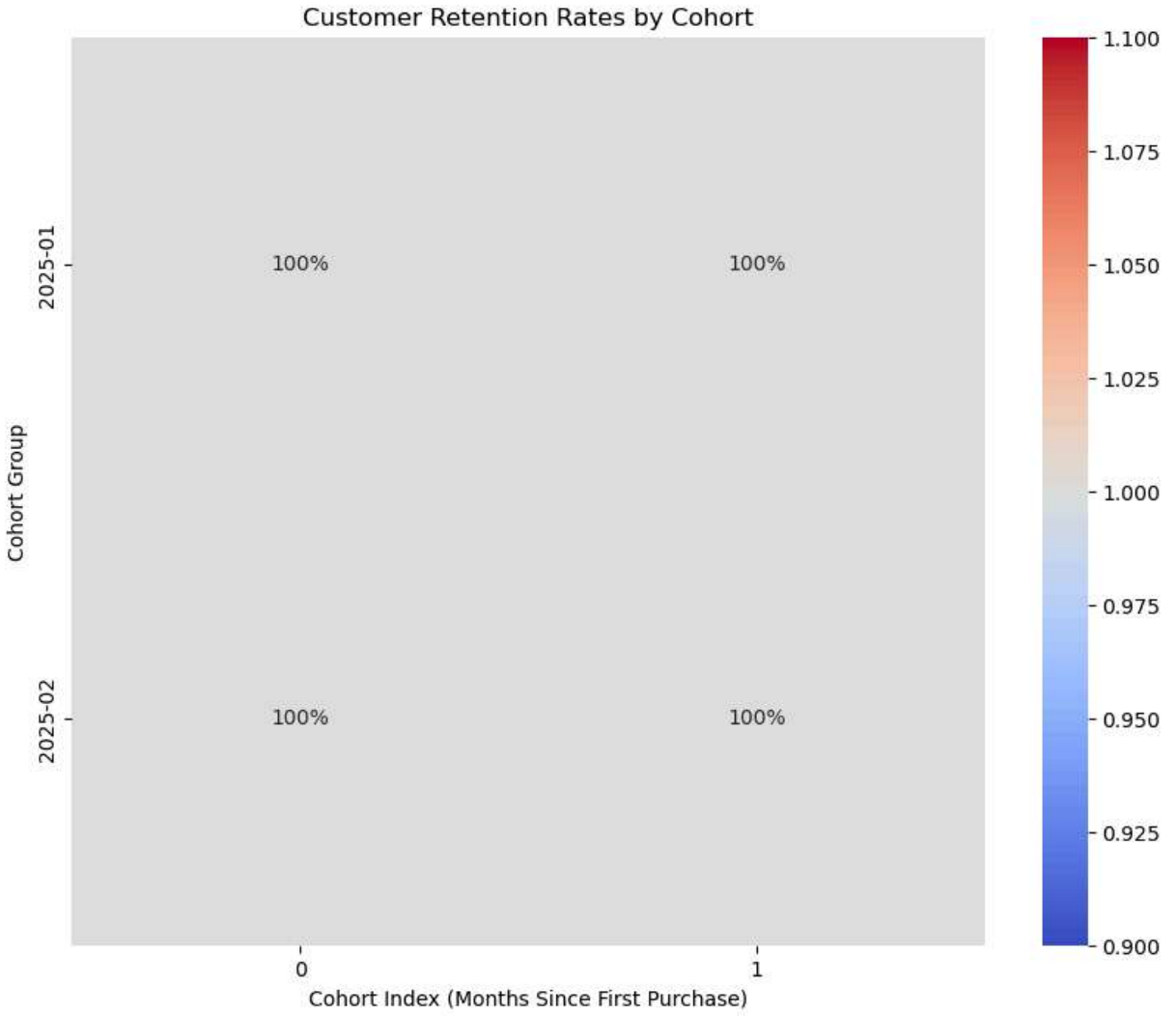
In [35]:
```
df['signup_month'] = pd.to_datetime(df['signup_date']).dt.to_period('M')
df['cohort_group'] = df.groupby('customer_id')['signup_month'].transform('min')
df['revenue_month'] = pd.to_datetime(df['revenue_date']).dt.to_period('M')
df['cohort_index'] = (df['revenue_month'].dt.year - df['cohort_group'].dt.year) * 1
```

```
cohort_counts = df.groupby(['cohort_group', 'cohort_index'])['customer_id'].nunique
cohort_sizes = cohort_counts[cohort_counts['cohort_index'] == 0][['cohort_group', '
cohort_sizes.rename(columns={'num_customers': 'cohort_size'}, inplace=True)

cohort_data = pd.merge(cohort_counts, cohort_sizes, on='cohort_group')
cohort_data['retention_rate'] = cohort_data['num_customers'] / cohort_data['cohort_
cohort_pivot = cohort_data.pivot(index='cohort_group', columns='cohort_index', valu

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
sns.heatmap(cohort_pivot, annot=True, fmt='.0%', cmap='coolwarm')
plt.title('Customer Retention Rates by Cohort')
plt.ylabel('Cohort Group')
plt.xlabel('Cohort Index (Months Since First Purchase)')
plt.show()
```



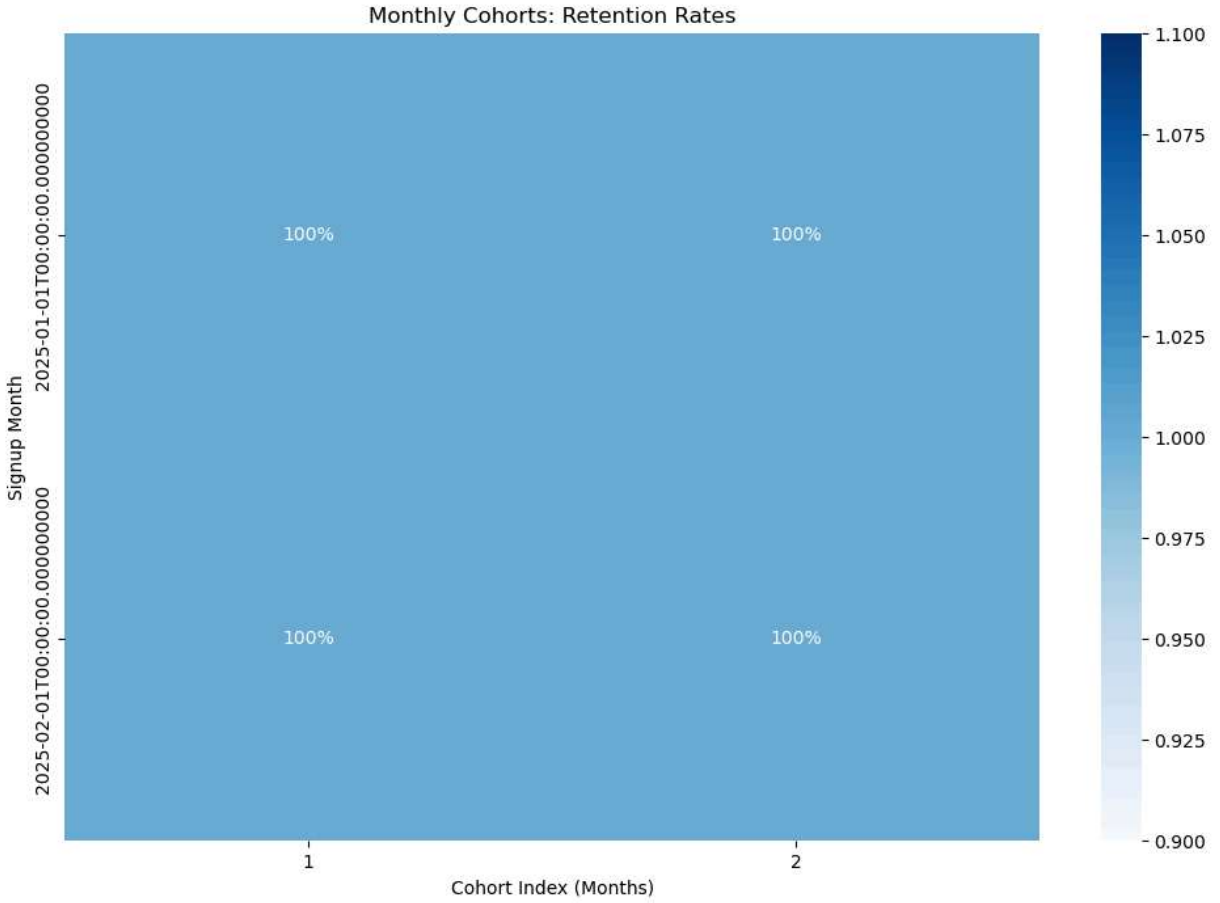

In [37]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt
df['signup_date'] = pd.to_datetime(df['signup_date'])
```

```
df['revenue_date'] = pd.to_datetime(df['revenue_date'])
def get_month(x):
    return dt.datetime(x.year, x.month, 1)
df['signup_month'] = df['signup_date'].apply(get_month)
df['revenue_month'] = df['revenue_date'].apply(get_month)
def get_date_int(df, column):
    year = df[column].dt.year
    month = df[column].dt.month
    return year, month

signup_year, signup_month = get_date_int(df, 'signup_month')
revenue_year, revenue_month = get_date_int(df, 'revenue_month')

df['cohort_index'] = (revenue_year - signup_year) * 12 + (revenue_month - signup_mo
cohort_data = df.groupby(['signup_month', 'cohort_index'])['customer_id'].nunique()
cohort_counts = cohort_data.pivot(index='signup_month', columns='cohort_index', val
cohort_sizes = cohort_counts.iloc[:, 0]
retention = cohort_counts.divide(cohort_sizes, axis=0)
plt.figure(figsize=(12, 8))
sns.heatmap(retention, annot=True, fmt='.0%', cmap='Blues')
plt.title('Monthly Cohorts: Retention Rates')
plt.ylabel('Signup Month')
plt.xlabel('Cohort Index (Months)')
plt.show()
```




In [39]:
```
df.to_csv("financial_kpisnew.csv", index=False)
```

In [41]:
```
df.to_excel("financial_kpisnew.xlsx", index=False)
```

In [43]:
```
import pandas as pd
import numpy as np

# Sample simulated monthly data
df = pd.DataFrame({
    'Month': pd.date_range(start='2024-01-01', periods=12, freq='MS'),
    'Revenue': np.random.randint(40000, 70000, 12),
    'Marketing_Expense': np.random.randint(8000, 15000, 12),
    'R&D_Expense': np.random.randint(5000, 10000, 12),
    'Infra_Expense': np.random.randint(2000, 5000, 12),
    'Salaries': np.random.randint(20000, 30000, 12),
    'Misc_Expense': np.random.randint(1000, 3000, 12)
})

# Total monthly expenses
df['Total_Expenses'] = df[['Marketing_Expense', 'R&D_Expense', 'Infra_Expense', 'Sa

# Burn Rate = Total Expenses - Revenue
df['Burn_Rate'] = df['Total_Expenses'] - df['Revenue']

# Convert Month to string for plotting
df['Month'] = df['Month'].dt.strftime('%Y-%m')

print(df[['Month', 'Revenue', 'Total_Expenses', 'Burn_Rate']])
```

```
      Month  Revenue  Total_Expenses  Burn_Rate
0   2024-01    65474           42844     -22630
1   2024-02    47495           50594       3099
2   2024-03    52868           48125      -4743
3   2024-04    57491           44360     -13131
4   2024-05    47636           46299      -1337
5   2024-06    48037           45990      -2047
6   2024-07    41924           46378       4454
7   2024-08    54058           51974      -2084
8   2024-09    63002           49435     -13567
9   2024-10    40743           50749      10006
10  2024-11    50802           53675       2873
11  2024-12    50525           48153      -2372
```

In [45]:
```
import pandas as pd
import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# 1. Generate monthly timeline
months = pd.date_range(start='2024-01-01', periods=12, freq='MS')

# 2. Simulated base data
data = {
    'Month': months,
    'Revenue': np.random.randint(40000, 70000, size=12),
    'Customers': np.random.randint(300, 600, size=12),
```

```
    'New_Customers': np.random.randint(40, 80, size=12),
    'Marketing_Expense': np.random.randint(8000, 15000, size=12),
    'R&D_Expense': np.random.randint(5000, 10000, size=12),
    'Infra_Expense': np.random.randint(2000, 5000, size=12),
    'Salaries': np.random.randint(20000, 30000, size=12),
    'Misc_Expense': np.random.randint(1000, 3000, size=12),
    'Cash_Balance': np.linspace(200000, 30000, 12)  # Simulate decreasing cash
}

df = pd.DataFrame(data)

# 3. Calculate derived KPIs
df['Total_Expenses'] = df[['Marketing_Expense', 'R&D_Expense', 'Infra_Expense', 'Sa
df['Burn_Rate'] = df['Total_Expenses'] - df['Revenue']
df['Cash_Runway_Months'] = (df['Cash_Balance'] / df['Burn_Rate'].abs()).round(1)
df['CAC'] = (df['Marketing_Expense'] / df['New_Customers']).round(2)
df['ARPU'] = (df['Revenue'] / df['Customers']).round(2)  # Avg Revenue per User
df['LTV'] = (df['ARPU'] * 12).round(2)  # Lifetime Value = ARPU * 12 months
df['LTV_to_CAC'] = (df['LTV'] / df['CAC']).round(2)
df['Gross_Margin'] = ((df['Revenue'] - df['Infra_Expense']) / df['Revenue']).round(

# 4. Revenue growth rate (%)
df['Revenue_Growth_Rate'] = df['Revenue'].pct_change().fillna(0).round(3) * 100

# 5. Format Month column
df['Month'] = df['Month'].dt.strftime('%Y-%m')

# 6. Final KPI selection
```

In [48]:

```
import pandas as pd
import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# 1. Generate monthly timeline
months = pd.date_range(start='2024-01-01', periods=12, freq='MS')

# 2. Simulated base data
data = {
    'Month': months,
    'Revenue': np.random.randint(40000, 70000, size=12),
    'Customers': np.random.randint(300, 600, size=12),
    'New_Customers': np.random.randint(40, 80, size=12),
    'Marketing_Expense': np.random.randint(8000, 15000, size=12),
    'R&D_Expense': np.random.randint(5000, 10000, size=12),
    'Infra_Expense': np.random.randint(2000, 5000, size=12),
    'Salaries': np.random.randint(20000, 30000, size=12),
    'Misc_Expense': np.random.randint(1000, 3000, size=12),
    'Cash_Balance': np.linspace(200000, 30000, 12)  # Simulate decreasing cash
}

df = pd.DataFrame(data)

# 3. Calculate derived KPIs
df['Total_Expenses'] = df[['Marketing_Expense', 'R&D_Expense', 'Infra_Expense', 'Sa
```

```
df['Burn_Rate'] = df['Total_Expenses'] - df['Revenue']
df['Cash_Runway_Months'] = (df['Cash_Balance'] / df['Burn_Rate'].abs()).round(1)
df['CAC'] = (df['Marketing_Expense'] / df['New_Customers']).round(2)
df['ARPU'] = (df['Revenue'] / df['Customers']).round(2)  # Avg Revenue per User
df['LTV'] = (df['ARPU'] * 12).round(2)  # Lifetime Value = ARPU * 12 months
df['LTV_to_CAC'] = (df['LTV'] / df['CAC']).round(2)
df['Gross_Margin'] = ((df['Revenue'] - df['Infra_Expense']) / df['Revenue']).round(

# 4. Revenue growth rate (%)
df['Revenue_Growth_Rate'] = df['Revenue'].pct_change().fillna(0).round(3) * 100

# 5. Format Month column
df['Month'] = df['Month'].dt.strftime('%Y-%m')

# 6. Final KPI selection
kpi_cols = [
    'Month', 'Revenue', 'Revenue_Growth_Rate', 'Customers', 'New_Customers',
    'Total_Expenses', 'Burn_Rate', 'Cash_Balance', 'Cash_Runway_Months',
    'CAC', 'LTV', 'LTV_to_CAC', 'Gross_Margin'
]
kpi_df = df[kpi_cols]

# 7. Save to CSV
kpi_df.to_csv("startup_kpis.csv", index=False)

# Optional display
print("📊 Startup KPI Dataset:")
print(kpi_df.head())
```

```
📊 Startup KPI Dataset:
     Month  Revenue  Revenue_Growth_Rate  Customers  New_Customers  \
0  2024-01    63654                  0.0        387             64
1  2024-02    55795                -12.3        399             66
2  2024-03    40860                -26.8        451             67
3  2024-04    45390                 11.1        430             55
4  2024-05    69802                 53.8        449             54

   Total_Expenses  Burn_Rate   Cash_Balance  Cash_Runway_Months     CAC  \
0           51694     -11960  200000.000000                16.7  163.05
1           48558      -7237  184545.454545                25.5  130.30
2           44858       3998  169090.909091                42.3  154.67
3           48600       3210  153636.363636                47.9  182.93
4           48988     -20814  138181.818182                 6.6  152.61

       LTV  LTV_to_CAC  Gross_Margin
0  1973.76       12.11          0.95
1  1678.08       12.88          0.94
2  1087.20        7.03          0.93
3  1266.72        6.92          0.90
4  1865.52       12.22          0.93
```

In [50]:
```
# Assume all New_Customers are part of a cohort for that month
cohort_df = df[['Month', 'New_Customers']].copy()
cohort_df.columns = ['Cohort_Month', 'Customer_Count']

# You could simulate retention like this:
```

```
cohort_df['Month_1'] = (cohort_df['Customer_Count'] * 0.9).astype(int)
cohort_df['Month_2'] = (cohort_df['Customer_Count'] * 0.75).astype(int)
cohort_df['Month_3'] = (cohort_df['Customer_Count'] * 0.6).astype(int)
# ... and so on

cohort_df.to_csv("cohort_retention.csv", index=False)
```

In [52]:
```
import pandas as pd

# Read the file manually, assuming it's a single column CSV
df = pd.read_csv('startup_kpis.csv', header=None)

# Split the single column by comma
split_df = df[0].str.split(",", expand=True)

# Set the first row as header
split_df.columns = split_df.iloc[0]
split_df = split_df.drop(0).reset_index(drop=True)

# Save clean version
split_df.to_csv('startup_kpis_fixed.csv', index=False)

print("✅ Fixed CSV saved as 'startup_kpis_fixed.csv'")
```

```
✅ Fixed CSV saved as 'startup_kpis_fixed.csv'
```

In [55]:
```
print(kpi_df.head())
```

```
     Month  Revenue  Revenue_Growth_Rate  Customers  New_Customers  \
0  2024-01    63654                  0.0        387             64
1  2024-02    55795                -12.3        399             66
2  2024-03    40860                -26.8        451             67
3  2024-04    45390                 11.1        430             55
4  2024-05    69802                 53.8        449             54

   Total_Expenses  Burn_Rate   Cash_Balance  Cash_Runway_Months     CAC  \
0           51694     -11960  200000.000000                16.7  163.05
1           48558      -7237  184545.454545                25.5  130.30
2           44858       3998  169090.909091                42.3  154.67
3           48600       3210  153636.363636                47.9  182.93
4           48988     -20814  138181.818182                 6.6  152.61

       LTV  LTV_to_CAC  Gross_Margin
0  1973.76       12.11          0.95
1  1678.08       12.88          0.94
2  1087.20        7.03          0.93
3  1266.72        6.92          0.90
4  1865.52       12.22          0.93
```

In [59]:
```
# Save the file properly
kpi_df.to_csv('startup_kpis1.csv', index=False)

print("✅ Data saved successfully!")
```

```
✅ Data saved successfully!
```

In [61]:
```
import pandas as pd
import numpy as np

np.random.seed(42)

months = pd.date_range(start='2024-01-01', periods=12, freq='MS')

data = {
    'Month': months,
    'Revenue': np.random.randint(40000, 70000, size=12),
    'Customers': np.random.randint(300, 600, size=12),
    'New_Customers': np.random.randint(40, 80, size=12),
    'Marketing_Expense': np.random.randint(8000, 15000, size=12),
    'R&D_Expense': np.random.randint(5000, 10000, size=12),
    'Infra_Expense': np.random.randint(2000, 5000, size=12),
    'Salaries': np.random.randint(20000, 30000, size=12),
    'Misc_Expense': np.random.randint(1000, 3000, size=12),
    'Cash_Balance': np.linspace(200000, 30000, 12)
}

df = pd.DataFrame(data)

# Calculate KPIs
df['Total_Expenses'] = df[['Marketing_Expense', 'R&D_Expense', 'Infra_Expense', 'Sa
df['Burn_Rate'] = df['Total_Expenses'] - df['Revenue']
df['Cash_Runway_Months'] = (df['Cash_Balance'] / df['Burn_Rate'].abs()).round(1)
df['CAC'] = (df['Marketing_Expense'] / df['New_Customers']).round(2)
df['ARPU'] = (df['Revenue'] / df['Customers']).round(2)
df['LTV'] = (df['ARPU'] * 12).round(2)
df['LTV_to_CAC'] = (df['LTV'] / df['CAC']).round(2)
df['Gross_Margin'] = ((df['Revenue'] - df['Infra_Expense']) / df['Revenue']).round(
df['Revenue_Growth_Rate'] = df['Revenue'].pct_change().fillna(0).round(3) * 100

df['Month'] = df['Month'].dt.strftime('%Y-%m')

kpi_df = df[['Month', 'Revenue', 'Revenue_Growth_Rate', 'Customers', 'New_Customers
             'Total_Expenses', 'Burn_Rate', 'Cash_Balance', 'Cash_Runway_Months',
             'CAC', 'LTV', 'LTV_to_CAC', 'Gross_Margin']]

# Save again
kpi_df.to_csv('startup_kpis.csv', index=False)
```

In [ ]: